

ANTLAB

ProofFlow AI Technical Specification

AI-Powered Pre-Launch Documentation Verification Platform for Web3 Teams — MVP
Architecture, Implementation Plan, and Cost Model

Executive Summary

ProofFlow AI is a pre-launch verification platform that automatically cross-references Web3 project documentation — whitepapers, GitHub repositories, smart contract ABIs, token distribution data, and website content — to detect inconsistencies, missing security signals, and investor-risk indicators before a project goes live. It produces a structured technical risk report consumed by founders, auditors, launchpads, and early-stage investors. The platform is designed to be built by a five-person team within 10 weeks on a \$40K–\$70K budget, assuming contract or equity-weighted compensation. Cash compensation structures require budget re-verification (see Cost Estimation section). ProofFlow reports are advisory triage outputs and do not constitute a security audit, financial advice, or a guarantee of contract safety. Emerging regulatory frameworks — including MiCA (effective December 2024) and the US GENIUS Act (July 2025) — are increasing documentation compliance expectations for token issuers, creating a direct tailwind for pre-launch verification tooling. Go-to-market distribution via launchpad and accelerator partnerships is a non-technical dependency not covered by the current five-person engineering team; BD resourcing is a pre-revenue requirement outside the MVP build scope (see Go-to-Market Strategy section).

Target market: smart contract audit services reached USD \$890M in 2024, growing at 22.8% CAGR to \$6.1B by 2033 (source: dataintelo.com); ProofFlow's addressable sub-segment is sized in the Market Sizing section with derivation logic

Based on publicly available feature documentation reviewed as of Q2 2025, no current competitor cross-references off-chain documentation against on-chain contract data before deployment; this assessment has a temporal boundary and requires ongoing competitive monitoring

MVP scope: single-project scan across five artifact types, outputting a severity-tagged risk report as web UI and downloadable PDF; Ethereum mainnet only

Primary competitive moat: the off-chain documentation cross-reference layer — not currently replicated by code-only tools such as GoPlus DeepScan (Q4 2025 beta) based on publicly available feature documentation; moat is subject to competitive response and is not structurally protected (see Risk Register)

ProofFlow's per-scan cost (\$500–\$2,000, estimated) represents a fraction of the cost of a comprehensive manual smart contract audit (\$20,000–\$500,000 per coinlaw.io), making it accessible as a pre-audit triage layer for early-stage projects; ProofFlow does not replace a manual audit — it surfaces issues before one is commissioned, reducing remediation cycles and wasted audit spend

CORE VALUE PROPOSITION

ProofFlow targets the documentation-consistency gap that existing automated tools leave entirely unserved: structured AI-driven cross-referencing of off-chain claims against each other and against on-chain data before any contract is deployed. All findings carry mandatory confidence scores and source citations; findings below defined confidence thresholds are flagged for human review before surfacing.

Problem Statement

Web3 projects routinely publish incomplete or internally inconsistent documentation before launch. Launchpads, auditors, and investors must manually reconcile whitepaper claims against contract architecture, tokenomics tables, and website copy — a process that consumes significant expert hours and remains error-prone. CertiK's Hack3D report documented approximately \$3.35B in losses across 630 security incidents in calendar year 2025 (source: PR Newswire / CertiK Hack3D Annual Report, published 2026, covering full-year 2025 incident data), reinforcing that pre-launch documentation failures translate directly into financial harm. The growing regulatory compliance burden introduced by MiCA (December 2024) and the US GENIUS Act (July 2025) further increases the cost of inadequate pre-launch documentation review for token-issuing projects.

All existing automated tools (Token Sniffer, De.Fi Scanner, GoPlus, SolidityScan) require a deployed contract address — none operate pre-deployment on documentation artifacts

Only approximately 2,000 blockchain security specialists exist globally (coinlaw.io), creating severe capacity constraints for manual review

No tool currently parses and cross-references whitepapers, GitHub READMEs, ABIs, and website marketing claims in a single pipeline — based on publicly available feature documentation reviewed as of Q2 2025

False or unverifiable tokenomics assumptions and undisclosed smart contract architecture divergences are leading investor-risk signals that go undetected until post-launch

CORE GAP AND EXPLICIT NON-GOALS

The pre-launch documentation-consistency layer is entirely unserved by automated tooling. ProofFlow targets this exact gap: structured AI-driven cross-referencing of off-chain claims against each other and against on-chain data before any contract is deployed. ProofFlow explicitly does NOT provide: a security audit, financial advice, real-time monitoring, multi-chain support (Ethereum mainnet only for MVP), Solana or non-EVM analysis, analysis of private GitHub repositories without granted access, or any guarantee of the absence of vulnerabilities.

Solution Architecture

ProofFlow operates as a three-layer asynchronous pipeline: Ingestion, Extraction and Embedding,

and Cross-Reference and Scoring. Each layer is decoupled via an async job queue (Celery + Redis) to handle variable document sizes and LLM API latency without blocking the user interface. The backend is Python FastAPI; the frontend is Next.js 14; persistent storage is PostgreSQL with pgvector for semantic similarity search. RAG orchestration uses LangChain (selected over LlamaIndex for its broader ecosystem of structured output parsers, native Pydantic integration, and larger community of Python FastAPI examples — decision required before Week 1 backend setup). LangChain manages prompt chains, retrieval pipelines, document chunking, and embedding calls; direct OpenAI and Anthropic SDK calls are routed through LangChain's LLM abstraction layer to enable model-switching without pipeline refactoring. No smart contract deployment is required for MVP. PDF reports are rendered via WeasyPrint (selected over Puppeteer for Python-native simplicity within the 10-week timeline; avoids headless Chromium dependency). Website scraping via Playwright respects robots.txt directives, enforces a minimum 2-second delay between requests, identifies itself with a disclosed user-agent string ('ProofFlowBot/1.0'), and is limited to publicly accessible pages; legal review of scraping jurisdiction compliance is required before beta launch for any EU-hosted project sites.

Layer 1 — Ingestion: parallel async fetching of whitepaper PDF (PyMuPDF with OCR fallback via LlamaParse for scanned documents), GitHub repo tree (GitHub REST API), Etherscan ABI and contract verification status (Ethers.js / Viem), token distribution JSON, and website text (Playwright scraper with robots.txt compliance and rate limiting)

Layer 2 — Extraction and Embedding: LangChain orchestrates LLM-based structured claim extraction per source using OpenAI GPT-4o or Anthropic Claude 3.5 Sonnet in JSON mode with Pydantic validation; claims stored with source chunk ID and attribution in pgvector; chunking strategy targets 512-token chunks with 64-token overlap to balance retrieval precision and LLM context efficiency

Layer 3 — Cross-Reference and Scoring: LangChain retrieval chain performs semantic similarity comparison across claim sets (cosine similarity threshold ≥ 0.75 for retrieval inclusion); rule-based contradiction detection supplements LLM findings; Slither static analysis runs conditionally on verified source only (see Slither scope note); all findings scored by severity and output as structured JSON rendered in the UI

Hallucination Mitigation and Confidence Scoring: (1) RAG retrieval — findings are only generated from document chunks with cosine similarity ≥ 0.75 to the query; chunks below this threshold are excluded from the context window and the finding is suppressed or flagged 'unverified'; (2) source citation grounding — each finding JSON object must include a `chunk_id` field referencing the pgvector row from which it was derived; LangChain's output parser enforces this field as non-nullable; any finding returned without a valid `chunk_id` is rejected by the post-processing validator and surfaced as 'unverified — source not located'; (3) confidence float computation — confidence is calculated as a weighted composite: $0.6 \times \text{retrieval cosine similarity score} + 0.4 \times \text{LLM self-consistency score}$ (the LLM is prompted twice with different temperature settings; agreement on the finding direction contributes 1.0, disagreement contributes 0.0 to the self-consistency component); (4) human review trigger — any critical-severity finding with confidence < 0.5 must render a 'Requires Manual Review' flag in the report UI and API response; these findings are surfaced in a separate review queue visible to the submitting user with a note that automated

confidence was insufficient for autonomous classification

Report delivery: async with email notification and webhook for API consumers; web UI + WeasyPrint PDF download for MVP

Confidence tiering and legal disclaimers are defined as mandatory schema fields; the confidence float must be computed from the specified method above and cannot be left as a default value; any finding with confidence below the defined threshold must be flagged for human review before surfacing

ARCHITECTURE DECISIONS

The vector DB (pgvector on the existing PostgreSQL instance) eliminates a separate Pinecone dependency for MVP, reducing infrastructure cost and operational surface area within the \$40K–\$70K budget constraint. Pinecone migration is a clean Phase 2 upgrade path if query volume demands it. Slither static analysis runs only when Etherscan source code is verified and downloadable; unverified contracts receive a 'Source unavailable — static analysis skipped' finding at medium severity. A 120-second subprocess timeout and error-handling wrapper are explicit implementation requirements; if Phase 2 timeline is at risk, Slither is demoted to optional MVP with a decision gate at the end of Week 5.

Smart Contract Design

ProofFlow does not deploy smart contracts for its MVP — it is a read-layer platform that queries existing on-chain data via Etherscan and EVM RPC endpoints. The 'Smart Contract Design' section therefore documents the on-chain data interfaces ProofFlow consumes and the ABI parsing module that structures contract data for cross-reference analysis. Future versions may introduce an on-chain report attestation registry, deferred to Phase 3.

Module	Function / Endpoint	Input	Output	Chain
ABI Fetcher	getContractABI(address)	Contract address + chain ID	Parsed ABI JSON with function signatures	Ethereum mainnet
Contract Verifier	checkVerificationStatus(address)	Contract address	Verified / unverified flag + source code URL	Ethereum mainnet
Token Supply Reader	getTokenomicsSnapshot(address)	ERC-20 contract address	Total supply, owner address, minting functions present	Ethereum mainnet
Slither Wrapper	runStaticAnalysis(sourceCode)	Verified Solidity source only; returns 'Source unavailable — static analysis skipped' (medium severity) if unverified; 120s timeout; subprocess isolated	Structured vulnerability list with severity tags	Off-chain (local)
Claim Comparator	crossReferenceABIsDocs(abiJson, claimsVector)	ABI JSON + embedded doc claims	Contradiction findings with confidence score and chunk_id citation	Off-chain (internal)

NO CONTRACT DEPLOYMENT REQUIRED

All MVP on-chain interactions are read-only calls to public Etherscan APIs and EVM JSON-RPC endpoints. This eliminates gas cost risk, audit liability for ProofFlow's own contracts, and deployment timeline pressure within the 10-week MVP window. Base, Arbitrum, and Polygon support are Phase 2 additions leveraging shared EVM tooling.

Backend and API Specification

The backend is a Python FastAPI application exposing a REST API for the frontend and an authenticated JSON report API for B2B consumers (launchpads, audit firms). Async job processing uses Celery with Redis as the message broker. All LLM calls are routed through a centralized LangChain LLM gateway module that enforces token budgeting, retry logic, and rate-limit handling —

this ownership falls on the backend developers given the absence of a dedicated MLOps engineer. Session tokens issued via NextAuth.js magic link expire after 24 hours; refresh is handled via NextAuth.js built-in JWT rotation; API key credentials for B2B consumers are stored server-side in environment variables managed via a secrets manager (Railway secret variables for MVP; AWS Secrets Manager for production scale). Production observability for the LLM pipeline is a Week 1 deliverable: all LangChain LLM calls log token consumption (prompt tokens, completion tokens, estimated cost at current model pricing) to a structured log stream; Prometheus metrics are exposed at /metrics for token usage per scan, Celery queue depth, and job failure rate; a Grafana dashboard (self-hosted on Railway) provides real-time visibility with an alert threshold triggering at 80% of the weekly LLM token budget. This is the primary mechanism for detecting inference cost overrun before it becomes critical.

```
# Core API endpoints (FastAPI)
POST    /api/v1/scans                # Submit new project scan (PDF + GitHub URL +
contract address)
GET     /api/v1/scans/{scan_id}     # Poll scan status and retrieve structured
report
GET     /api/v1/scans/{scan_id}/pdf # Download WeasyPrint-rendered PDF report
POST    /api/v1/webhooks/register   # Register callback URL for async report
delivery
GET     /api/v1/health              # Liveness + dependency status check
GET     /metrics                    # Prometheus token usage, queue depth, job
failure metrics

# Report JSON schema (top-level)
{
  "scan_id": "uuid",
  "project_name": "string",
  "overall_risk_score": 0-100,
  "scan_completed_at": "ISO8601 timestamp",
  "findings": [
    {
      "category": "string",
      "severity": "critical|high|medium|low|info",
      "description": "string",
      "source": "string",
      "chunk_id": "uuid",          # pgvector row reference - non-nullable
      "confidence": 0.0-1.0,      # computed per Hallucination Mitigation spec
      "requires_manual_review": true|false # true if critical + confidence <
0.5
    }
  ],
  "disclaimer": "string"          # legal disclaimer text - mandatory field
}

# PostgreSQL core schema (simplified)
CREATE TABLE scans (
  id UUID PRIMARY KEY,
```

```

    project_name TEXT,
    status TEXT, -- queued|ingesting|extracting|analyzing|scoring|complete|failed
    created_at TIMESTAMPTZ,
    completed_at TIMESTAMPTZ,
    overall_risk_score INT
);
CREATE TABLE findings (
    id UUID PRIMARY KEY,
    scan_id UUID REFERENCES scans(id) ON DELETE CASCADE,
    category TEXT,
    severity TEXT,
    description TEXT,
    source TEXT,
    chunk_id UUID REFERENCES document_chunks(id),
    confidence FLOAT NOT NULL,
    requires_manual_review BOOLEAN NOT NULL DEFAULT false
);
CREATE TABLE document_chunks (
    id UUID PRIMARY KEY,
    scan_id UUID REFERENCES scans(id) ON DELETE CASCADE,
    source_type TEXT, -- whitepaper|github|abi|tokenomics|website
    content TEXT,
    embedding VECTOR(1536) -- pgvector
);
CREATE TABLE api_keys (
    id UUID PRIMARY KEY,
    user_id UUID,
    key_hash TEXT, -- HMAC-SHA256 of raw key; raw key never stored
    created_at TIMESTAMPTZ,
    last_used_at TIMESTAMPTZ,
    rate_limit_per_hour INT DEFAULT 10
);

```

CRITICAL IMPLEMENTATION NOTE

Every finding in the JSON schema carries a non-nullable 'confidence' float (0.0–1.0), a 'chunk_id' source citation, and a 'requires_manual_review' flag. These are mandatory from day one — LLM-generated findings consumed by auditors and investors create real-world liability if unqualified. Legal disclaimer text must be reviewed by external counsel before any beta access is granted to launchpad or auditor partners. Environment separation: development, staging, and production environments are maintained as separate Railway projects with isolated secret variable sets; no production secrets are present in development or staging configurations.

Frontend Integration

The frontend is a Next.js 14 application using the App Router and Tailwind CSS. It provides a project submission form, a real-time scan progress tracker (polling every 5 seconds with WebSocket upgrade path for Phase 2), and a structured report viewer with severity-filtered finding cards and a downloadable PDF export rendered server-side via WeasyPrint. The designer and one full-stack developer own this workstream; it runs in parallel with backend pipeline development from Week 3 onward using mocked report JSON. Authentication uses NextAuth.js magic link (email OTP) for MVP; session tokens expire after 24 hours with automatic JWT rotation via NextAuth.js built-in refresh; API keys for B2B consumers are stored as HMAC-SHA256 hashes in PostgreSQL — raw keys are displayed once at creation and never stored in plaintext.

Submission form: three required inputs — whitepaper PDF upload (max 50MB), GitHub repository URL, EVM contract address (optional pre-deployment); optional website URL and token distribution JSON

Progress UI: five-stage status indicator (Ingesting → Extracting → Analyzing → Scoring → Ready) with estimated time remaining; findings flagged 'Requires Manual Review' rendered in a distinct review queue visible to the submitting user

Report viewer: findings grouped by category (documentation completeness, tokenomics consistency, contract alignment, public claims, security signals) with severity badges, confidence scores, source citations, and chunk-level traceability links

PDF export: server-rendered via WeasyPrint; branded report with legal disclaimer footer on every page; disclaimer text to be approved by legal counsel before beta

Authentication: NextAuth.js magic link for MVP with 24-hour session expiry; API key management UI for B2B consumers showing key creation, last-used timestamp, and revocation

DESIGN PRIORITY

The report viewer is the highest-value screen in the product — it is the deliverable that launchpad operators and investors interact with directly. All design effort should prioritize clarity of severity hierarchy, confidence score visibility, and source traceability over aesthetic polish in the MVP. Findings requiring manual review must be visually distinct from autonomous findings to prevent misinterpretation.

Security Considerations

ProofFlow's security surface has two distinct dimensions: platform security (protecting the infrastructure and user data) and output integrity (ensuring AI-generated findings are not misleading or exploitable by adversarial project submissions). Both must be addressed at MVP launch, not deferred. The absence of a dedicated QA or security testing role means the smart contract specialist and AI engineer must jointly own adversarial input testing. PDF sandbox isolation uses Docker containers with a read-only filesystem, no network access, CPU limit of 2 cores, memory limit of 512MB RAM, and a 60-second processing timeout. Malformed PDFs that cause PyMuPDF to raise

an exception return a 'Parse failed — manual upload required' finding at medium severity; this fallback is an explicit implementation requirement. The sandbox specification must be confirmed in the infrastructure setup document by the backend developer before Week 2.

LLM hallucination mitigation: all findings require a valid chunk_id citation to a pgvector source chunk; the confidence float is computed as specified in the Solution Architecture section ($0.6 \times$ retrieval cosine similarity + $0.4 \times$ LLM self-consistency score); ungrounded claims are suppressed or flagged 'unverified' — never presented as autonomous findings; critical findings with confidence < 0.5 surface a 'Requires Manual Review' flag

Adversarial input handling: projects may submit polished decoy documentation designed to pass automated checks; the system must surface absence-of-evidence signals (e.g., ABI present but no GitHub code, tokenomics claimed but no on-chain distribution verifiable); adversarial test suite validates this behavior (see Testing Strategy)

PDF parsing attack surface: uploaded PDFs are processed in an isolated Docker sandbox container (read-only filesystem, no network, 2 CPU / 512MB RAM limit, 60s timeout); no execution of embedded scripts; file size capped at 50MB for MVP; malformed PDF exception fallback returns 'Parse failed — manual upload required' at medium severity

Website scraping compliance: Playwright scraper respects robots.txt directives, enforces minimum 2-second inter-request delay, discloses user-agent ('ProofFlowBot/1.0'); legal review of jurisdiction-specific scraping legality required before beta launch for EU-hosted project sites

API authentication: all report API endpoints require HMAC-signed API keys (SHA-256); raw keys stored as hashes only; rate limiting enforced at 10 scans per hour per key to prevent cost-drain attacks on LLM budget

Data handling: whitepaper PDFs and extracted claims stored encrypted at rest (AES-256); retention policy of 90 days; user-initiated deletion removes the source document, all associated document_chunks rows (including pgvector embeddings), and all findings rows via CASCADE delete; submitted documents are not used for model training or fine-tuning without explicit user opt-in

LIABILITY BOUNDARY AND PRIVACY COMMITMENTS

ProofFlow's reports are advisory triage outputs, not security audits or financial advice. Legal disclaimers must appear on every report page and in every API response. This language must be reviewed by external legal counsel before the platform accepts any beta users. The platform commits to publishing a privacy policy before beta launch. ProofFlow operates as a data processor under GDPR for any project documents containing personal data submitted by EU-based users; data processor obligations, including documented deletion mechanisms and third-party sub-processor disclosure, apply from the first beta submission.

Testing Strategy

Given the absence of a dedicated QA role, testing responsibilities are distributed: the AI/LLM engineer owns pipeline accuracy and hallucination regression tests; the smart contract specialist owns ABI parsing and Slither output validation; the backend developers own API integration, async job reliability, and observability tests; the designer owns UI acceptance testing. A minimum viable test suite must gate each weekly sprint delivery. The labeled document corpus (10 known-inconsistent Web3 project document sets) is created and labeled jointly by the AI engineer and smart contract specialist in Week 2; labeling decisions are documented in a shared annotation log so the corpus can be independently reviewed. The precision target of ≥ 0.80 for critical-severity findings is an internally agreed MVP acceptance threshold — no external industry benchmark is available for this tool category; if this threshold is not met at Week 9, a defined response plan triggers: the AI engineer has one week (Week 10) for prompt revision before beta launch is delayed. Adversarial test pass/fail threshold: the system must surface at least one absence-of-evidence or contradiction signal in 4 of 5 crafted document sets; crafted sets are created by the smart contract specialist using real project documentation with deliberate introduced inconsistencies.

Unit tests: claim extraction accuracy against a labeled corpus of 10 known-inconsistent Web3 project document sets (whitepapers with deliberately mismatched tokenomics and ABIs, labeled by AI engineer + smart contract specialist in Week 2); target precision ≥ 0.80 for critical-severity findings (internally agreed MVP threshold)

Integration tests: end-to-end scan submission through to report delivery for three canonical project archetypes — DeFi protocol, NFT platform, and DAO governance token; success criterion: scan completes in under 10 minutes for a 50-page whitepaper with zero critical pipeline failures

Adversarial tests: submit five crafted document sets (created by smart contract specialist with deliberate inconsistencies) designed to evade detection; pass criterion: system surfaces at least one absence-of-evidence or contradiction signal in 4 of 5 sets

Load tests: simulate 20 concurrent scan submissions to validate Celery queue stability and LLM rate-limit handling without job failure; LLM token consumption logged and reviewed against weekly budget threshold

PDF output tests: validate that every generated WeasyPrint report PDF contains required disclaimer text and that all finding severity labels, confidence scores, and 'Requires Manual Review' flags render correctly across Chrome, Firefox, and Safari

LLM OUTPUT REGRESSION PROTOCOL

Any change to the LLM prompt templates or model version must be preceded by a regression run against the labeled document corpus using the LangChain prompt versioning log. A drop of more than 5 percentage points in precision for high or critical findings constitutes a blocking failure requiring prompt revision before deployment. Prompt versions are tagged in Git alongside the model identifier used; regression results are recorded in a shared test results document updated after each run.

Deployment Roadmap

The 10-week MVP roadmap is structured as three phases. Infrastructure and ingestion pipeline work begins Week 1 to protect against the highest-identified risk: any delay in vector DB setup, LLM API procurement, LangChain configuration, or async job queue setup compresses delivery time for all downstream pipeline work. Production observability setup (Prometheus + Grafana on Railway) is a Week 1 deliverable, not a post-launch addition, to enable real-time LLM cost monitoring from the first test scan. Frontend development runs in parallel from Week 3 using mocked data. No scope creep into real-time monitoring, wallet analysis, or multi-chain support is permitted within this timeline. Three environment tiers are maintained throughout: development (local Docker Compose), staging (Railway project with separate secret variable set), and production (Railway project with isolated secrets). Secrets are managed via Railway secret variables for MVP; no secrets appear in .env files committed to version control. Pilot outreach targeting 3+ launchpad and auditor candidates (Polkastarter, DAO Maker, TrustPad, and two audit firms as named targets) begins in Week 1; BD is owned by a founding team member outside the five-person engineering team; goal of confirming 1–2 beta participants by Week 8, with internal beta delivery by Week 10. If no BD owner is confirmed by Week 2, this represents a project-level risk requiring escalation (see Risk Register).

Phase	Weeks	Deliverables	Owner
Phase 1 — Foundation	1–3	FastAPI skeleton, Celery+Redis job queue, pgvector setup, LangChain configuration and RAG pipeline scaffold, Prometheus + Grafana observability stack, LLM API keys procured, PDF ingestion (PyMuPDF + Docker sandbox + OCR fallback), GitHub API ingestion, Etherscan ABI fetcher, Next.js project scaffold, Week 1 observability live	Both backend devs + AI engineer
Phase 2 — Pipeline and Analysis	4–7	LangChain claim extraction pipeline with Pydantic schema, confidence float computation implemented, chunk_id citation enforcement, cross-reference engine (cosine similarity $\geq$	AI engineer + smart contract specialist + designer

0.75 + rule-based),
Slither integration
(conditional on verified
source; decision gate
at Week 5 if timeline at
risk), severity scoring
model, report JSON
schema finalized,
frontend report viewer
(mocked data), labeled
corpus created and
annotated

Phase 3 — Integration
and Launch

8–10

End-to-end scan flow
live, WeasyPrint PDF
report export, HMAC
API key authentication,
webhook delivery,
adversarial test suite
executed, legal
disclaimer review
completed, privacy
policy published, 1–2
beta partners
confirmed (outreach
began Week 1),
production
deployment on
Railway (prod
environment),
environment
separation verified

Full team — QA
owned by backend
devs + smart contract
specialist

TIMELINE RISK AND RISK REGISTER

The 10-week schedule is achievable only with strict scope discipline. Ethereum mainnet is the sole supported chain for MVP. Base, Arbitrum, and Polygon are Phase 2 additions leveraging shared EVM tooling. Solana and non-EVM chains are explicitly deferred to Phase 3 or later.

Risk Register (key items): (1) LLM inference cost overrun — Likelihood: H, Impact: H, Mitigation: token budget enforced in LangChain gateway from Week 1, Prometheus alert at 80% weekly budget, Owner: backend lead; (2) Partner acquisition failure — Likelihood: M, Impact: H, Mitigation: BD outreach begins Week 1 with named targets, escalation if no confirmation by Week 6, Owner: founding team BD lead; (3) Hallucination liability event — Likelihood: M, Impact: H, Mitigation: confidence scoring + human review flags + legal disclaimers + legal counsel review before beta, Owner: AI engineer + legal; (4) GoPlus adds documentation layer — Likelihood: M, Impact: M, Mitigation: accelerate launchpad lock-in and proprietary corpus development, competitive monitoring monthly, Owner: product lead; (5) Team compensation structure invalidates budget — Likelihood: L, Impact: H, Mitigation: confirm cash vs. equity split with each team member before Week 1, Owner: founding team; (6) Slither integration timeline slip — Likelihood: M, Impact: L, Mitigation: decision gate at Week 5, demote to optional MVP if at risk, Owner: smart contract specialist; (7) Website scraping legal exposure — Likelihood: L, Impact: M, Mitigation: robots.txt compliance enforced, legal review before EU beta, rate limiting from launch, Owner: backend lead.

Cost Estimation

The \$40K–\$70K budget covers 10 weeks of five-person team compensation plus infrastructure and API costs. Budget confidence is moderate: LLM inference costs are the highest variable and depend heavily on average whitepaper length and chunking efficiency. LLM cost assumptions: average whitepaper = 50 pages $\approx$ 25,000 tokens; with 512-token chunks, approximately 50 chunks per document; extraction and cross-reference passes consume approximately 5,000 tokens per scan at GPT-4o pricing (\$0.005/1K input, \$0.015/1K output as of Q2 2025); estimated cost per scan $\approx$ \$0.08–\$0.25; 200–400 scans during the 10-week build and test period yields \$16–\$100 in inference cost at low volume, scaling to \$1,500–\$4,000 under adversarial load testing and iteration — this estimate could be 3–5x higher if whitepapers are longer than assumed or chunking is inefficient; token budgets per scan must be enforced in the LangChain gateway from Week 1. The team compensation assumption is contract or equity-weighted; if any team member is fully salaried cash, this budget does not cover 10 weeks at market rates and runway must be re-verified with the founding team before work begins.

Workstream	Low Estimate (USD)	High Estimate (USD)	Notes
Team compensation (5 people, 10 weeks)	\$28,000	\$48,000	Assumes contract / partial equity; verify cash vs. equity split before Week 1
LLM API costs (OpenAI GPT-4o or Claude 3.5 Sonnet)	\$1,500	\$4,000	Assumes avg 50-page whitepaper, ~5,000 tokens/scan, 200–400 test scans; could be 3–5x higher under adversarial load testing if assumptions are wrong; enforce token budget in LangChain gateway from Week 1
Infrastructure (Railway/Render, Redis, PostgreSQL, storage, Grafana)	\$400	\$1,200	Scales with scan volume; pgvector avoids separate Pinecone cost for MVP; includes Prometheus + Grafana observability stack
Third-party APIs (Etherscan Pro, GitHub, Playwright hosting)	\$300	\$800	Etherscan Pro recommended for rate limits; GitHub public API free tier sufficient
Legal disclaimer review + scraping compliance review	\$500	\$2,000	External counsel review of report disclaimer language and Playwright scraping jurisdiction compliance before beta launch
Design tools, WeasyPrint renderer, miscellaneous	\$300	\$800	Figma, WeasyPrint (Python-native, no Chromium dependency), domain, SSL

BUDGET GUIDANCE AND BUSINESS MODEL

Total estimated range: \$31,000–\$56,800, fitting within the \$40K–\$70K constraint at both ends with contingency headroom. The single largest risk to budget is LLM inference cost overrun — token budgets per scan must be enforced in the LangChain gateway from Week 1, monitored via Prometheus with an 80%-of-weekly-budget alert, and reviewed weekly by the backend lead. Business Model (indicative estimates — all figures require validation with paying customers): Free tier: 1 scan per project (lead generation); Pro tier: \$299/month for up to 20 scans (early-stage founders); API tier: \$150 per scan for launchpad B2B API access (target: 3+ launchpad contracts); Enterprise: custom contract for accelerators and audit firms. Estimated gross margin at API tier: scan cost \$0.08–\$0.25 LLM + \$0.10 infrastructure vs. \$150 revenue — margin >99% at assumed volume before team cost allocation. 12-month revenue scenario (indicative, unverified): Low — 2 launchpad API contracts × 10 scans/month × \$150 = \$3,600/month ARR; Mid — 5 contracts × 20 scans/month × \$150 = \$15,000/month ARR; High — 10 contracts + 50 Pro subscribers × \$299 = \$29,950/month ARR. These are illustrative estimates only; actual conversion rates and scan volumes are unvalidated. A Market Sizing note: ProofFlow's addressable pre-launch tooling sub-segment is estimated at \$150M–\$300M ARR opportunity by 2027 (internal analyst estimate, unverified; derivation: approximately 5,000–10,000 token project launches per year on target chains × \$150–\$300 average scan revenue × 20–30% addressable fraction × 3-year compounding growth assumption). This figure has no third-party basis and should not be cited externally without independent validation. The sourced anchor figure is the \$890M smart contract audit market in 2024 (dataintelo.com).